第 2 章 数学与模型基础

> 本章目标：不推公式、不做证明，只建立十个"直觉"。学完之后，你看 AI 文档里的术语不再是黑话，也能判断一个说法是不是在糊弄你。

2.0 先说清楚：哪些不用背

本课**不要求**你推导任何公式，不需要会求导、会矩阵乘法、会写损失函数。你需要的只是"听到一个词，脑子里能浮出一幅画"。

下面每个概念都按同一格式讲：**一句话是什么 → 一个具体例子 → 这和你用 AI 有什么关系。**

2.1 向量与相似度

一句话：向量就是一串数字，可以理解成空间里的一个点（或一支箭头）；相似度看的是两支箭头方向是否一致，而不是长度。

例子：三句话变成三个向量后，"今天天气不错"和"外面阳光很好"方向接近，"今天天气不错"和"帮我修一下这个 bug"方向几乎垂直。常用的度量叫**余弦相似度（cosine similarity）**，取值越接近 1 越像，接近 0 表示无关，负数表示方向相反。

与你有关：所有"语义检索""知识库问答""去重"背后都是这件事。你给 Agent 挂一个文档库，它先在库里找与问题最相似的几段——找错了，后面全错，而且模型会一本正经地基于错材料回答。

2.2 矩阵：变换，而不是表格

一句话：矩阵是对空间的一次"变换"——旋转、拉伸、压扁、投影。

例子：一张图片在程序里就是一大块数字。所谓"把图片旋转 90 度""把彩色图转成灰度图"，本质都是乘一个特定的矩阵。神经网络的一层，也基本就是"乘一个矩阵 + 加一点非线性"。

与你有关：模型"参数有多少"说的就是这些矩阵里的数字总数。听到"这个模型有几十亿参数"，意思是他有几十亿个可以被调整的数字——不是几十亿条规则。

2.3 卷积：拿一个小窗口扫过整张图

一句话：卷积就是用同一个很小的窗口（卷积核）在整张图上来回滑动，每个位置算一次"这里像不像某个图案"。

例子：一个 3×3 的窗口学到的是"左上到右下的斜线"。它滑过整张图，就在每个位置报告"这里有没有这条斜线"。下一层把这些斜线组合成拐角，再往上组合成零件轮廓。

关键好处：同一个窗口在图上所有位置复用（**参数共享**），所以参数少、且"图案出现在左上还是右下都认得出"（**平移不变性**）。

与你有关：机器人视觉里检测工件、找抓取点，绝大多数是卷积网络。它快的根本原因就在这里——不是因为它"更聪明"。

2.4 梯度下降：蒙眼下山

一句话：不知道谷底在哪，就用脚探一下哪个方向最陡，朝那个方向迈一小步，重复很多次。

> 类比：**蒙着眼站在山坡上下山。** 你只能感知脚下附近的坡度，于是每次朝最陡的下坡方向走一步。步子迈得合适，就能走到低处；迈太大，可能一步跨过谷底冲到对面山坡；迈太

小，走到天黑还在半山腰。



这个"坡度"在数学上叫**梯度（gradient）**，"迈一步"叫一次**更新**。上千次更新之后，损失就降下来了。

与你有关：所有训练过程都是这件事的变体。你看到的"训练 loss 曲线不降""loss 变成 NaN"，基本都是步子或地形出了问题（学习率、数据、初始化），而不是模型"想不通"。

2.5 学习率：步子大小

一句话：学习率（learning rate）就是每一步迈多大。

学习率	现象	直觉
太小	损失缓慢下降，训练很久也没收敛	一小步一小步挪，走到天黑
合适	前期快速下降，后期平缓	正常下山
太大	损失剧烈震荡，甚至越来越大、变成 NaN	一步跨过谷底，越跳越远

与你有关：微调模型时最常见的失败原因就是学习率。别人的配方能跑，换到你的数据上炸了，先怀疑这个。

2.6 损失函数：给"错了多少"打分

一句话：损失函数（loss function）是一个打分器，把模型的输出和正确答案一比，给出一个数，越小越好。

例子：分类任务里，模型说"这是猫，置信度 0.9"，而答案是猫，损失就很小；如果说"这是狗，置信度 0.9"，损失就很大。**训练的全部目的就是让这个分数变小。**

与你有关：损失函数决定了模型"在乎什么"。如果打分时只奖励"答得快"、不奖励"答得对"，模型就会学会又快又错。设计任务时想清楚打分器，比选模型更重要。

2.7 过拟合与欠拟合：背题 vs 没学会

> 类比：**过拟合就像把习题册答案全背下来的学生**——原题全对，换个数字就懵；**欠拟合就像上课没听懂的学生**——连原题都做不对。

	训练集表现	测试集表现	原因
欠拟合	差	差	模型太简单，或训练不够
刚好	好	好	学到了规律
过拟合	极好	差	把噪声和个例也记住了

怎么发现：拿一部分数据不参与训练，专门用来"考"。训练分数一路飙升、考试分数却开始变差，就是过拟合的信号。

与你有关：给 Agent 写提示词、准备示例时同理。塞进去十个高度相似的样例，它可能只会照抄格式，遇到稍微不同的输入就崩。**样例要覆盖变化，不要覆盖重复。**

2.8 参数与超参数

一句话：**参数**是模型自己学出来的（权重）；**超参数**是人定的（层数、学习率、批大小、训练轮数）。

	谁决定	举例	训练中会变吗
参数	数据 + 优化	权重矩阵里的每个数字	会
超参数	人	学习率、网络层数、批大小	不会

与你有关：调参（tuning）这个词经常被滥用。改提示词不叫调参，改学习率才叫调超参数——搞清楚自己在动哪一类旋钮，能省掉大量瞎试。

2.9 Epoch 与 Iteration

一句话：**iteration（迭代）**是更新一次参数；**epoch** 是把整个训练集完整过一遍。

例子：训练集有 1000 条样本，批大小（batch size）为 10，那么一个 epoch 需要 100 次迭代。

与你有关：看到"训练了 3 个 epoch"就知道数据被反复看了三遍——遍数太多容易过拟合，太少容易欠拟合。这是个很实用的判断刻度。

2.10 Embedding：把词变成坐标

一句话：embedding 是给每个词（或图、或物品）分配一组坐标，让"意思相近的东西坐标也相近"。

例子：在训练好的坐标空间里，"猫"和"狗"离得近，"猫"和"汽车"离得远。有时还能看出方向关系，比如"国王 − 男人 + 女人 ≈ 女王"这种组合规律。

怎么来的：没人手工标注"猫和狗近"。坐标是训练过程中"为了让下一个词预测得更准"顺带学出来的副产品。

与你有关：RAG（检索增强）的整条链路都建在这上面：切片 → 转成向量 → 存库 → 按相似度召回。检索效果差，八成是切片或 embedding 模型选错了，而不是大模型不行。

2.11 激活函数：给网络加"弯"

一句话：激活函数（activation function）在线性变换之后加一个非线性弯折，让网络能拟合曲线而不只是直线。

例子：最常用的 **ReLU** 规则极简——负数变 0，正数不变。就这么一条，堆叠上千层之后，网络就能逼近非常复杂的函数。

为什么必须要：如果每层都只是"乘矩阵加偏置"，叠一百层也等价于一层，再深也没用。**非线性是深度有意义的前提。**

与你有关：看到"ReLU / GELU / SiLU"这些名字，只需知道它们是不同的弯折方式，属于实现细节，不影响你理解整体。

2.12 张量：多维数组的统称

一句话：张量（tensor）就是任意维度的数组，是这些框架里所有数据的统一容器。

维度	叫法	例子
0 维	标量	一个数字：3.14
1 维	向量	一句话的 embedding

维度	叫法	例子
2 维	矩阵	一张灰度图
3 维	三阶张量	一张彩色图（高×宽×通道）
4 维	四阶张量	一批彩色图（批×高×宽×通道）

与你有关：PyTorch 报错里最常见的就是形状不匹配（shape mismatch）。看懂"这个张量是几维、每维多大"，你就能自己排查大半的报错，而不是把错误信息整段丢给 AI 让它猜。

2.13 一个完整的心理图像

把这些串起来，训练就是：

1. 数据变成**张量**；
2. 张量经过若干"**矩阵**变换 + **激活函数**"（如果是图像，其中一些是**卷积**）；
3. 输出和答案一比，算出**损失**；
4. 用**梯度下降**、按**学习率**调一小步**参数**；
5. 一个 **epoch** 里重复很多次 **iteration**；
6. 用没参与训练的验证集检查有没有**过拟合**。

> 要点：这六步是所有深度学习训练的骨架。第五章之后的任何新名词，都只是在这六步里换零件，不是换骨架。

课后题

1. （概念）用自己的话解释"过拟合"，并举一个你在用 AI 写代码时可能遇到的对应现象。
2. （概念）区分参数与超参数，各举两个例子。再说明"调整提示词"属于哪一类。
3. （动手）假设训练集有 6000 条样本、批大小为 32，问一个 epoch 大约有多少次迭代？如果要训练 5 个 epoch，参数一共被更新多少次？
4. （动手）找一篇介绍某个开源模型的文档，找出里面提到的：参数量、上下文长度、训练用了多少数据。判断哪些是参数、哪些是超参数、哪些是训练配置。
5. （思考）"损失降得很低就说明模型很好"这句话有什么问题？结合 2.6 和 2.7 说明。
6. （思考）如果给一个模型的学习率设得极大，训练会发生什么？请用"蒙眼下山"的类比描述过程，并解释为什么最终数值可能变成 NaN。